

用 Claude Code 蓋一座 AI 軟體工廠

把 Claude skills、sub-agents 與守門機制制度化成一條
可版控、可重複、可交接的 AI 開發產線 —— 成功經驗與踩坑全紀錄

資料截止 2026-06-04 · 所有數字基於 git 紀錄、`.claude/` 設定與 `docs/` 文件實測查證 · 按 **S** 開講者備註

分享結構（六段）

1 · 背景說明

為什麼要把 AI 開發流程化？團隊與專案背景。

2 · 專案內容

11 週產出數據、三層文件治理。

3 · 使用工具

.claude 生態系：skills / agents / pipeline / 測試 / 守門。

4 · 成功經驗分享

值得繼續投入的做法 · 為什麼有效 · 成效 · 關鍵成功因素。

5 · 失敗 / 踩坑經驗

沒達預期的嘗試 · 原因與教訓 · 重來會怎麼做。

6 · 額外分享

有價值的發現、新工具推薦、參考資源。

第 4、5 段是本場核心 —— 誠實講哪些值得複製、哪些坑別再踩。

PART 1

背景說明

我們是誰、做什麼，以及為什麼決定把「人怎麼帶 AI」這件事制度化。

專案與團隊背景

專案：PredictMarket

- 二元預測市場（是/否）**aPaaS** 平台，提供可嵌入的預測市場基礎設施
- 核心機制：LMSR 撮合、自動結算、費率控制在 10% 內
- 技術棧：Laravel 12 + Lighthouse GraphQL · React 19 + TS · MySQL 8 · Filament 後台
- MVP 已上線：`prototype.kf-test.com`

團隊規模

- 核心開發 1-2 人，11 週維持高產出
- 全程以 GitHub Issue / Project Board 驅動
- 合作規範、流程、安全守門 **全部寫進 repo** (`.claude/` + `docs/`)
- 關鍵問題：小團隊如何用 AI 規模化，又不犧牲品質與可交接性？

為什麼要把 AI 開發「流程化」？

痛點：沒有流程的 AI 開發

- ✗ 每次 prompt 結果不一致、品質看運氣
- ✗ AI 不懂專案硬規則 → 違反架構、洩密、breaking change
- ✗ 規格在腦袋裡 → 無法審查、無法交接
- ✗ 「先這樣之後再改」的技術債堆積
- ✗ 前後端各做各的 → 介面對不上

我們的選擇：制度化

- ✓ 規範寫進 `.claude/rules/` → 每次自動載入
- ✓ 流程寫進 skill → 每次照同一條 pipeline 走
- ✓ 規格落地成 `docs/specs/` → 可審、可追、可交接
- ✓ 守門靠 hooks + 權限 → 機器強制，不靠自律
- ✓ 前後端由同一份 **SPEC** 切分並行 → 介面天生對齊

PART 2

專案內容

11 週實際產出了什麼，以及支撐這些產出的三層文件治理。

11 週產出數據快照

2026-03-13 ~ 2026-06-04，約 11 週

405

main commits

87.7%

commit 綁定 Issue
(355 / 405)

99

SPEC (平均 ~9/週)

13 / 5

RFC / ADR

程式碼規模

層	檔案	行數
後端 PHP (app/)	598	~65.8k
前端 TS/TSX	232	~40.8k
GraphQL schema	1	2,223 (唯一真相源)
Services / Migrations /	64 / 149 / 218	檔

節奏 (commits / 月)

月份	commits	
2026-03	108	
2026-04	163	
2026-05	115	
2026-06	19	(至 6/4)

穩定高產出，非一次性衝刺。Issues 已開到 #506、PR 到 #508。

三層文件治理：決策 → 架構 → 實作

RFC

決策層

要不要做、規則是什麼

13 份



ADR

架構層

技術選型怎麼定

5 份



SPEC

實作層

怎麼寫 code

99 份



Code

實作

PR + Review

405 commits

SPEC 的固定七段結構

摘要 · 需求規格(含 user story/UI flow) · 資料與狀態(DB/GraphQL/狀態機) · 實作切分 · 驗收標準 · 測試情境 · 風險待釐清

關鍵設計：Blast Radius + 強制分層

每份 SPEC 必填影響範圍 (Backend / Frontend / GraphQL / Infra)，實作切分強制分 `### Backend / ### Frontend`，讓並行 agent 精準切任務。

PART 3

使用工具

把流程、規範、守門都寫進 `.claude/` —— 六個互相咬合的元件。

.claude/ 生態系全景

① Skills (12 個 · 2,196 行)

可重複呼叫的「流程腳本」。1 個主編排 + 11 個專職工站。

② Sub-agents (4 個 · 全用 Opus)

規格作者 / 獨立審查 / 後端 / 前端，各自獨立 context。

③ Rules (5 檔 · 404 行)

依檔案路徑自動載入的硬規則：Laravel / GraphQL / React / TS / 安全。

④ Hooks (2 腳本 + 4 內嵌)

機器強制守門：擋 .env 外洩、編輯後自動 lint、PHP 自動格式化。

⑤ 三層 Memory

使用者偏好 / 每個 agent 的累積學習 / 專案硬規則。

⑥ Docs + Permissions

RFC/ADR/SPEC 文件治理 + 破壞性指令白/黑名單 (43 allow / 9 deny)。

六個元件互相咬合 —— 這就是「AI 軟體工廠」的廠房配置圖。

12 個客製 Skills

Skill	角色	行	分類
spec-then-build	一條龍主流程：規格 → 並行實作 → 驗證	341	🎯 編排
filament-admin	後台 Filament 開發規範與既有模式	455	🔗 領域指南
testing	PHPUnit + Vitest 單元/功能測試標準	310	✅ 測試
b2b-api	B2B REST API 開發規範（HMAC/文檔同步）	198	🔗 領域指南
security-audit	OWASP / GraphQL 安全 / 依賴漏洞掃描	166	🔒 安全
e2e-test / review-code / review-spec	Playwright E2E · PR 審查 · SPEC 審查	108/105/87	✅ 🔍 測試/審查
session-back / workspace-config	恢復脈絡 · 管理 .claude 設定本身	93/174	⚙️ 元工具
debug-troubleshoot / db-migration	結構化除錯 · Laravel migration 流程	80/79	🐛 🔗 維運/指南

+ 原生內建 skills (code-review / security-review / verify / deep-research ...) 可與客製 skill 組合使用。

關鍵洞察：skills 是被「編排」的

spec-then-build 像主程式，沿途條件式呼叫其他 skill 當副程式

主流程內部呼叫

階段	呼叫
Stage 1.5 審查	review-spec / spec-reviewer
Stage 2 後端（條件式）	filament-admin / b2b-api / db-migration
Stage 3 / 3.5	testing / e2e-test
Stage 4 品質門	review-code + security-audit

獨立入口

- **session-back** — 開工時恢復脈絡
 - **debug-troubleshoot** — 卡關時除錯
 - **workspace-config** — 演進這套系統時
- = 可組合系統：主幹串起專職工站，也能單獨拿出來用。

主流程 spec-then-build：五階段產線



適用中大型、跨前後端功能；小修改不必走全程。

4 個 Sub-agents + 三層記憶體

Agent	職責	模型	權限 / 記憶
product-spec-writer	需求 → 結構化 SPEC	Opus	可寫 · project memory
spec-reviewer	獨立審查、挑盲點	Opus	唯讀 · 無記憶（防偏見）
laravel-graphql-backend	後端實作	Opus	可寫 · project memory
senior-react-ts-graphql	前端實作	Opus	可寫 · project memory

L1 使用者記憶

偏好 / 回饋 / 專案脈絡，跨對話持久。

L2 Agent 記憶

每個 agent 累積的專案教訓（如「通知表名 user_notifications」）。

L3 Rules 規範

依路徑自動載入的硬規則。

測試策略：五層金字塔

不靠單一手段，層層補網 —— 越上層越像真實使用者

層	手段	本專案現況（實測）	抓什麼
L1	PHPUnit Unit	22 檔	單元邏輯 / Service
L2	PHPUnit Feature	173 檔（後端共 195 測試檔）	API / 整合行為
L2'	Vitest（前端元件）	32 檔	React 元件
L3	Playwright E2E	49 spec · 11 個 @critical	使用者黃金路徑
L4	AI 探索性測試 🤖	20 scenarios · 79 份報告	腳本抓不到的混亂路徑
L5	Prod smoke	規劃中（另議）	上線後監測

L3 守底線、L4 主動探雷 —— 下一頁深入 L4。

L4 — AI 探索性測試 (自製工具)

是什麼 / 為什麼

- 一個 **AI agent** 自主駕駛瀏覽器探索 app，找腳本化測試抓不到的 bug
- E2E 只跑「黃金路徑」；AI 會亂序下注、每步切語系、中途打斷
- 正式記錄於 **RFC-010** (已 Accepted)

怎麼跑 (架構)

- Claude Agent SDK 驅動 multi-turn 工具迴圈
- 注入 **14 個工具**：navigate / click / fill / screenshot / read_console / read_network / assert_invariant / report_anomaly ...
- 後端 **Invariant API** (`/internal/invariants/check`，正式環境 404) 驗：機率和=1 · 抽水≤10% · 資金守恆 · 無孤兒交易
- 守門：local-only、**不排程**、**不上 CI**、**不自動開 Issue**、預算上限 (單 run \$0.50 / 總 \$5)

守門機制：機器強制，不靠自律

Hooks (工具呼叫層攔截)

- **block-env-read.sh** — Read/Bash 前擋下任何想印出 `.env` 的操作 → 密鑰不外洩
- **lint-after-edit.sh** — 每次編輯後自動 `tsc --noEmit / php -l`
- `artisan test` 前清 GraphQL schema 快取
- 對話結束自動 `pint --dirty` 格式化改動的 PHP

Permissions (白/黑名單)

- ✓ **Allow (43 條)** : `npm / tsc / vitest / playwright`、docker 內 `artisan`、`git`、`gh` ... 日常免打斷
- ⚡ **Deny (9 條)** : `rm -rf` · `git push --force` · `reset --hard` · `migrate:fresh` · `DROP TABLE/DATABASE` ...

關鍵差異：規範不是「希望 AI 記得」，而是系統在工具呼叫層攔截——想違反也做不到。

PART 4 · 本場核心

成功經驗分享

哪些做法值得繼續投入資源？為什麼有效、成效如何、關鍵成功因素是什麼？

✓ 值得繼續投入的六件事

不分技術棧 —— 這六件事是這套流程真正產生槓桿的地方

紀律

Issue-first + commit 綁定

所有工作從 Issue 開始，**87.7% commit 綁 #N**。任何一行 code 都能反查「為什麼改」。

契約

Schema-first 契約先行

先改 GraphQL schema 再寫 resolver、禁 breaking change。**介面先定死**，前後端才能並行不打架。

對抗式驗證

獨立審查員 + grep 反查

派**唯讀、無記憶**的 spec-reviewer 挑盲點；主程式再逐項 grep 驗證引用 method 是否真存在，擋 AI 幻覺。

並行設計

Blast Radius + 強制分層

SPEC 先填影響範圍、實作切分強制分 BE/FE。**先講清楚動到哪幾層**，並行才安全。

機器護欄

守門機器化

hook 在**工具呼叫層**攔密鑰、9 條 deny 封破壞性指令。想違反也做不到。

主動探雷

AI 探索測試 + 不變量 API

讓 AI 亂玩抓混亂路徑、後端驗帳，配 **\$ 上限、不自動開 issue** 守門。

加碼小招：git stash 分辨新舊測試失敗 · commit 但不自動 push · 三層 memory 累積教訓

為什麼有效？成效如何？關鍵成功因素

為什麼有效

- **品質前移**：規格期就抓問題，比上線後便宜
- **去偏見**：獨立審查 + grep 反查，不讓作者審自己
- **不靠自律**：護欄機器強制，疲勞也不會破口
- **介面先定**：schema-first 讓並行天生對齊

成效（量化 + 質化）

- 11 週 **99 SPEC + 405 commits**、**1-2 人**維持
- **87.7%** commit 綁定 Issue
- AI 探索測試**抓到 @critical 漏掉的真實 bug** (#216)
- 可重複 / 可審查 / **可交接**（流程在 repo 非腦中）

關鍵成功因素

- **規範入庫**：硬規則寫進可版控的 rules
- **流程成腳本**：重複流程寫成 skill
- **human gate**：關鍵節點人類核可、不自動 push
- **誠實文化**：不盲信 AI、用 deterministic 補驗

一句話：把「人怎麼帶 AI」從隱性知識，變成可版控、可重複、可交接的工程資產。

PART 5 · 本場核心

失敗 / 踩坑經驗

哪些嘗試沒達預期或撤回？原因與教訓是什麼？如果重來一次會怎麼做？

真正「做了又撤回 / 失敗」的嘗試

① 時區對齊當 hotfix (撤回)

嘗試：為修 HOT 分數時間 bug，想直接改 DB 連線時區對齊 +8。

失敗原因：226 欄全 `TIMESTAMP`，改連線時區會位移既有資料 +8h 成 split-brain —— 這是 **ADR 級資料遷移**，不能當 hotfix。#476

教訓：有些「修正」本質是資料遷移，硬幹會擴大災難；做了又撤回。正解：收斂共用 parse util (#488 / #506)。

② Codex MCP 架構 (Round 1 中止)

嘗試：用 MCP 架構接 OpenAI Codex 訂閱跑探索測試。

失敗原因：Round 1 MCP 架構三個 blocker 卡死，**中止重來**。

教訓：Round 2 改**內嵌 HTTP server** 架構繞過全部 blocker、recall 3/3 成功 (RFC-012)。先做小 spike 驗證架構，別一路走到底。

× 開發中踩過的坑（附 issue 佐證）

測試環境

harness 硬鎖 SQLite

隱藏 MySQL 專屬 bug：#476 分數溢位、Filament groupBy 撞 ONLY_FULL_GROUP_BY 1055。聚合/list 頁需用 MySQL 跑 E2E。

E2E 偽綠

條件式 skip 的 render-race

等錯 locator 再 count 會偶發誤判空集合，**silent skip** 核心路徑（#494）。先等目標 locator 可見再判斷。

AI 假陰性

盲信「0 anomaly」

探索 runner 不存 GraphQL body，AI 報「沒問題」可能假陰性。0 異常必須用 deterministic curl 補驗。

主題 token

Tailwind @theme inline + hex

@theme inline + hex literal 會把 utility 編譯死，runtime 切主題無效。token 用 hex 必須無 inline，E2E 驗 computed-style。

金流原子性

balance 操作未自包 transaction

incrementBalance/decrementBalance 自身不開 transaction，caller 必須自包；多補完成路徑須共用單一 lock + idempotent 方法。

流程 / 人

流程過重 · bus factor · spec drift

全套 pipeline 套小修改反而慢；產出集中單人；code 演進後 SPEC 會脫節需定期對帳。

共同根因：外部假設（DB 引擎 / 時區 / locator / AI 自評）都要在「目標 scope」內精確驗證，不可單一信號下結論。

如果重來一次，會怎麼做？

測試與驗證

- ✓ 測試 harness 從第一天就用 **MySQL** 跑關鍵聚合 / list 測試，不靠 SQLite
- ✓ E2E 條件式 skip 一律先等目標 **locator** 可見再判斷
- ✓ AI 探索結果預設不信任，deterministic curl 補驗納入標準流程

架構與流程

- ✓ 時區 / 金額等基礎決策一開始就立 **ADR** + 共用 util，別等踩雷
- ✓ 新整合先做小 **spike** 驗架構再投入（學 Codex Round 2）
- ✓ 明訂「輕量 vs 全程」準則，並早點讓第 2、3 人實跑分散 bus factor

核心心法：把「驗證外部假設」變成流程的預設動作，而不是踩雷後的補救。

PART 6

額外分享

有價值的發現、新工具推薦，以及可複用的方法論與參考資源。

有價值的發現與新工具推薦



發現：AI 探索測試真的有效

- PoC 6 天 / 9 run 抓到下注送出鈕被 **viewport** 截掉的 UX bug —— 既有 @critical 完全沒抓到 (#216)
- 成本極低：6 天約 **\$3.27** (上限 \$10)
- 關鍵：搭配後端**不變量 API** 才能驗「帳有沒有算錯」，不只是點得動

✂ 新工具推薦：雙 runner 控成本

- **Anthropic API key** (預設 · production-grade)
- + **OpenAI Codex 訂閱** (成本固定) —— spike 驗證後採「並存」
- ~36 個 codex run 另抓到多個 finding；訂閱制讓探索測試的邊際成本趨近 0

推薦給有大量 agent 探索/批次任務的團隊：用**訂閱制 runner** 跑可重複的高頻任務、用 **API runner** 跑需要穩定品質的關鍵任務。

可複用方法論 + 參考資源

核心原則（跨技術棧通用）

- ① 規範入庫：硬規則寫進可版控的 rules
- ② 流程成腳本：重複流程寫成 skill
- ③ 規格先行：先產可審查的規格再實作
- ④ 獨立審查：派無偏見的 agent 挑盲點
- ⑤ 守門機器化：用 hooks / 權限強制

導入順序：CLAUDE.md → 幾條 rules → 擋密鑰 hook → 權限名單 → 第一個 skill → multi-agent。

參考資源

- **Claude Agent SDK** — 驅動探索 runner 的 multi-turn 工具迴圈
- 本專案文件：RFC-010 (AI 探索測試) · RFC-012 (雙 runner) · ADR-002 (主題 token)
- 內建 skills：code-review / security-review / verify / deep-research
- 守門範本：`.claude/hooks/block-env-read.sh` 可直接複用

五個原則跨語言、跨框架通用 —— 我們用 Laravel + React，換成任何棧邏輯相同。

一句話帶走

我們把「人怎麼帶 AI 寫好程式」
變成了可版控、可重複、可交接的工程資產。

99

SPECs

12 / 4

Skills / Agents

5 層

測試金字塔



AI 自主探索測試

Q & A · 查證資料來源：git 紀錄、`.claude/` 設定檔、`docs/` 文件 (2026-06-04)